

### 0.0.1 Question 1.a: Modeling Resource Usage

What is the equation for the amount of pepper needed to manufacture  $x_1$  and  $x_2$ . What is the equation for the amount of vinegar? (Use [Mathpix](#) to write equations)

#### SOLUTION

The equations for each ingredient are as follows

$$\begin{cases} y_p = 5x_1 + 7x_2 \\ y_v = 4x_1 + 2x_2 \end{cases}$$

For vinegar, the total amount used in manufacturing Kapatio and Zriracha is

$$y_v = 4x_1 + 2x_2$$



### 0.0.2 Question 1.b1: Resource Usage vs. Total Resource Constraint

Total amount of pepper needed cannot exceed total available. Write down the inequality expressing this relationship. Do the same for vinegar. These inequalities are your resource constraints. Additionally, variables  $x_1$  and  $x_2$  are non-negative: i.e. amount of manufactured goods cannot be negative.

Rewrite the system of constraint inequalities into a matrix inequality:  $Ax \leq b$ , where  $x = (x_1, x_2)^T$ . Arrange rows of  $A$  and  $b$  such that:

- Row 1: total pepper amount constraint
- Row 2: total vinegar amount constraint
- Row 3: Kapatio non-negativity constraint
- Row 4: Zriracha non-negativity constraint

Less than symbol in  $Ax \leq b$  means element-wise.

### SOLUTION

Given total amount of pepper needed cannot exceed total available. We can generate inequality relationship based on the system of equations before.

$$\begin{cases} 5x_1 + 7x_2 \leq 30 \\ 4x_1 + 2x_2 \leq 12 \end{cases}$$

The non-negativity constraint ensures that the amount of Kapatio and Zriracha produced is non-negative, such that

$$\begin{cases} -x_1 \leq 0 \\ -x_2 \leq 0 \end{cases}$$

In matrix form

$$\begin{bmatrix} 5 & 7 \\ 4 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \leq \begin{bmatrix} 30 \\ 12 \end{bmatrix}$$

and

$$\begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \leq \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Hence let  $x = (x_1, x_2)^T$ , then the above equation can be defined as

$$\begin{bmatrix} 5 & 7 \\ 4 & 2 \\ -1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \leq \begin{bmatrix} 30 \\ 12 \\ 0 \\ 0 \end{bmatrix}$$

### 0.0.3 Question 1.c: Feasible Region Boundary

In a list named `boundary`, create four data frames for each equality in  $Ax = b$ . These lines indicate where the feasible area ends. Set column names as

- $x_1$
- $x_2$
- constraints

Note the use of latex codes (feel free to use [Mathpix](#)).

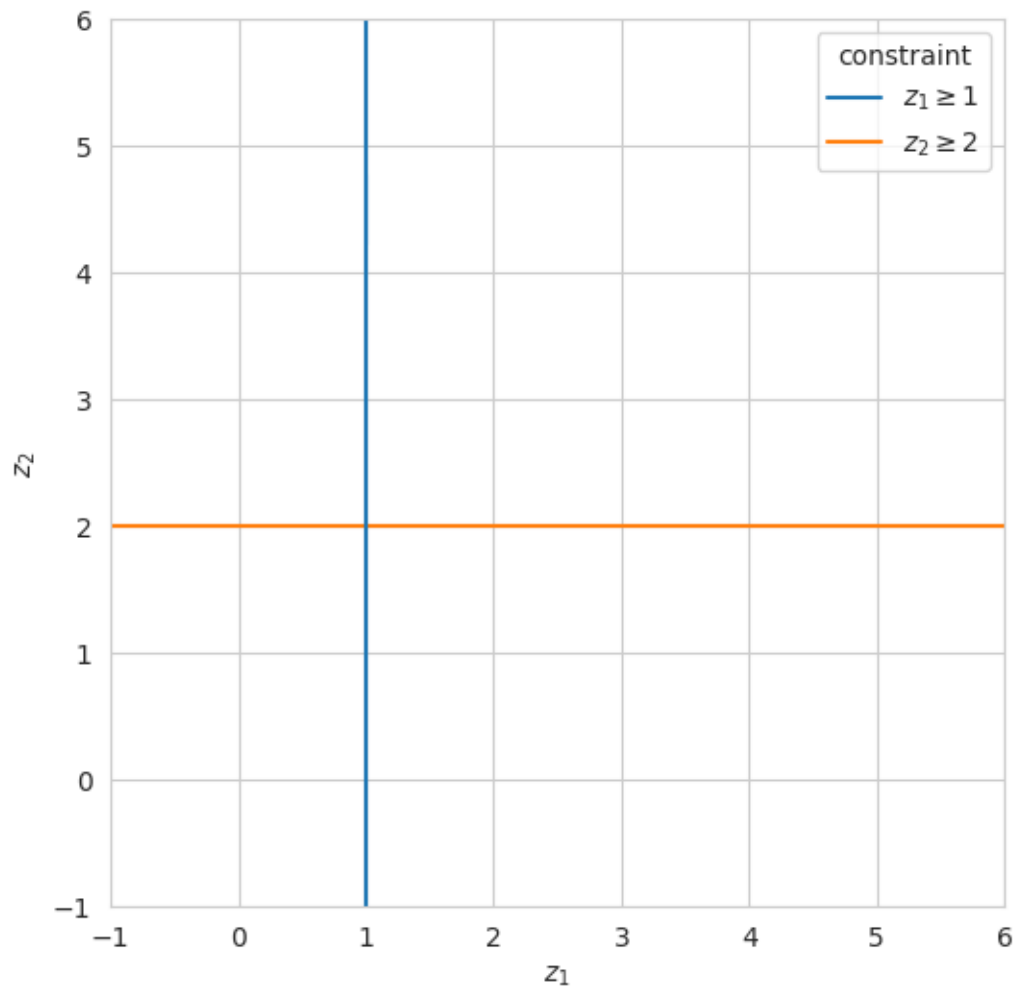
**Toy Example: Drawing Boundaries** Here is a **toy example** of drawing two constraint boundaries by constructing data frames:

```
In [6]: z1_line = np.linspace(-1, 10, 500)
        z2_line = np.linspace(-1, 10, 500)

        boundary = [
            pd.DataFrame({
                '$z_1$': np.ones_like(z2_line)*1,
                '$z_2$': z2_line,
                'constraint': '$z_1 \geq 1$'
            }),
            pd.DataFrame({
                '$z_1$': z1_line,
                '$z_2$': np.ones_like(z1_line)*2,
                'constraint': '$z_2 \geq 2$'
            }),
        ]
        fig, ax = plt.subplots(figsize=(6, 6))
        sns.lineplot(x='$z_1$', y='$z_2$', hue='constraint', data=pd.concat(boundary).reset_index(), ax=
        plt.xlim(-1, 6)
        plt.ylim(-1, 6)
        plt.show()

/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
if pd.api.types.is_categorical_dtype(vector):
```

```
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated
  with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated
  with pd.option_context('mode.use_inf_as_na', True):
```



Sometimes, things just do not work as expected.

In the toy example code,

```
sns.lineplot(x='$z_1$', y='$z_2$', hue='constraint', data=pd.concat(boundary), ax=ax).axvline(1)
```

what seems strange about the plotting command? Why was the strange code necessary?

**SOLUTION**





By dissecting the command below and reading the documentation, report what each of the following lines does:

- `((y1_grid >= 1) & (y2_grid >= 2)).astype(int)` (What is the output of running this command?)
- `origin='lower'`
- `extent=(y1_grid.min(), y1_grid.max(), y2_grid.min(), y2_grid.max())`
- `cmap='Greys'`
- `alpha=0.3`
- `aspect='equal'`

## SOLUTION

`((y1_grid >= 1) & (y2_grid >= 2)).astype(int)`: the output will have the same shape as `y1_grid` and `y2_grid`, with values 1 where both conditions (`y1_grid >= 1` and `y2_grid >= 2`) are met and 0 elsewhere.

`origin='lower'`: set the origin (0,0) point at the bottom-left corner of the plot.

`extent=(y1_grid.min(), y1_grid.max(), y2_grid.min(), y2_grid.max())`: sets the minimum and maximum values of `y1_grid` for the horizontal extent and `y2_grid` for the vertical extent.

`cmap='Greys'`: sets the colormap for the plot using `grayscale colormap`

`alpha=0.3`: sets the transparency level of the plot to be 70% transparent.

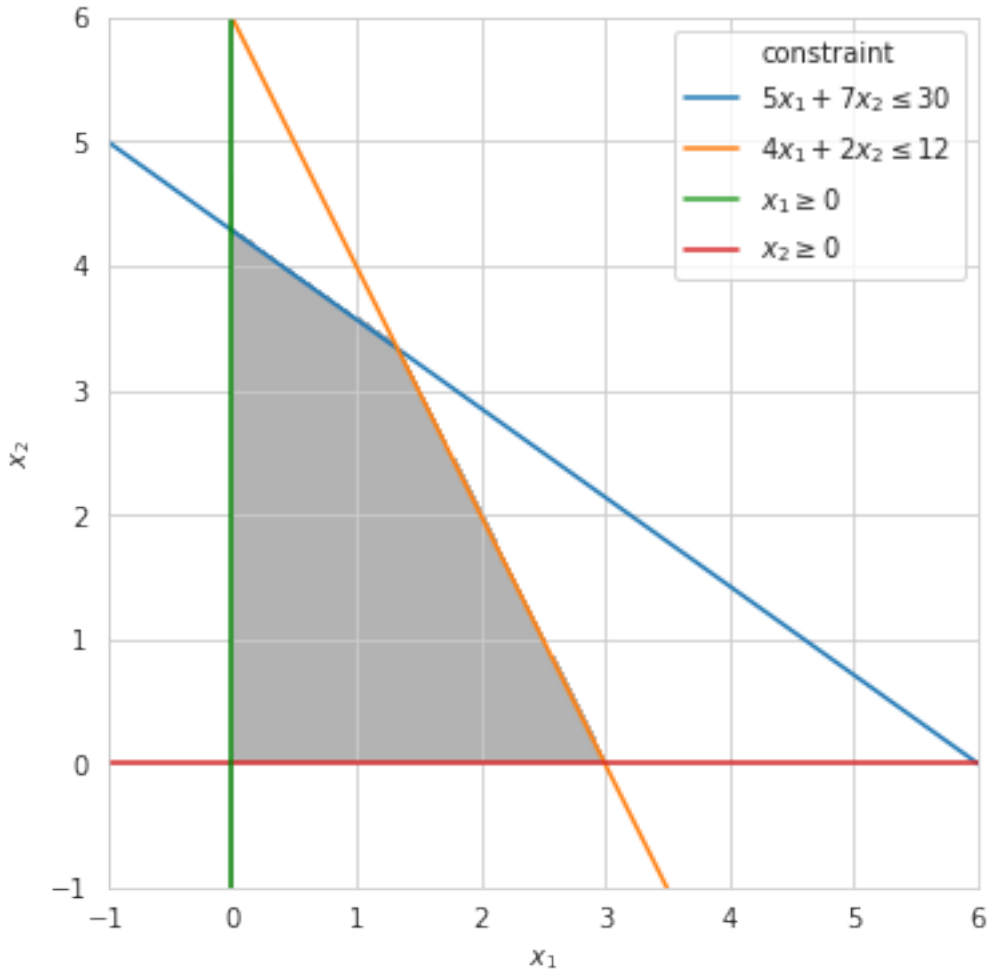
`aspect='equal'`: sets that one unit on the x-axis is the same length as one unit on the y-axis.



#### 0.0.4 Question 1.e: Visualizing the Feasible Region

Finally, create a figure that shows constraint boundaries and the interior region shaded with a light grey color.

Your output will look like this:



```
In [11]: fig, ax = plt.subplots(figsize=(6, 6))
x1_grid, x2_grid = np.meshgrid(x1_line, x2_line)

ax.imshow(
    ((x1_grid >= 0) & (x2_grid >= 0) & (5 * x1_grid + 7 * x2_grid <= 30) & (4 * x1_grid + 2 * x2_grid <= 12)),
    origin='lower',
    extent=(x1_grid.min(), x1_grid.max(), x2_grid.min(), x2_grid.max()),
    cmap="Greys", alpha = 0.3, aspect='equal')

# ax = sns.lineplot(???).axvline(???)
ax = sns.lineplot(
```

```

    data=hull,
    x='$x_1$',
    y='$x_2$',
    hue='constraint' # Using the 'constraint' column to label different lines
).axvline(0, color='green')

#ax.axvline(x=0, color='green')

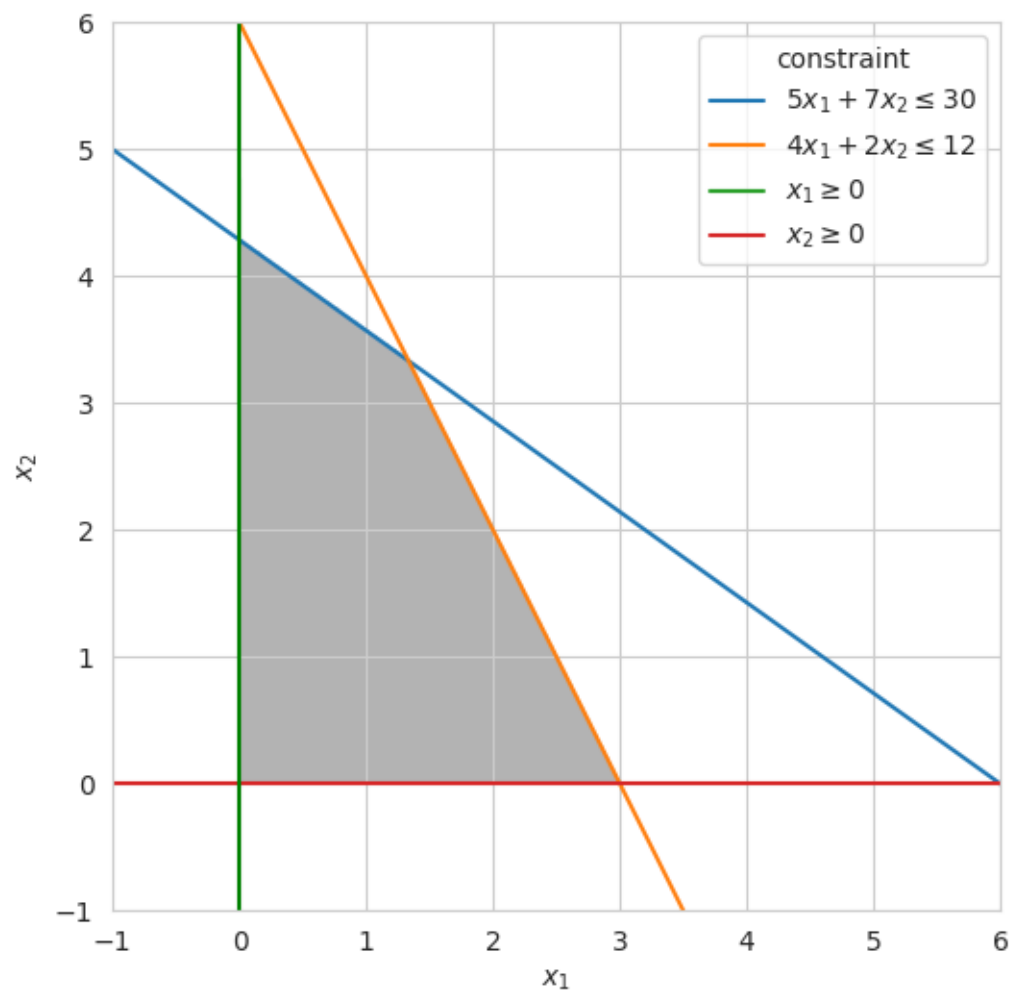
plt.xlim(-1, 6)
plt.ylim(-1, 6)
plt.show()

```

```

/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
    if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
    if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
    if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
    if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
    if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is
    with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is
    with pd.option_context('mode.use_inf_as_na', True):

```





In the context of linear programming,  $Ax \leq b$  is called the *feasible region* (including the appropriate sections of the boundaries). Denote the (shaded) feasible region as set  $C$ . Points  $(x_1, x_2) \in C$  satisfy all of the constraints.

Describe in plain words the feasible region in the context of hot sauce manufacturing. Specifically, which constraint is violated (if any) by a point at:

- $(x_1, x_2) = (4, 1)$
- $(x_1, x_2) = (0, 5)$
- $(x_1, x_2) = (3, 4)$

### **SOLUTION**

$(4, 1)$  violates  $4x_1 + 2x_2 \leq 12$

$(0, 5)$  violates  $4x_1 + 2x_2 \leq 12$  and  $5x_1 + 7x_2 \leq 30$

$(3, 4)$  violates  $4x_1 + 2x_2 \leq 12$  and  $5x_1 + 7x_2 \leq 30$





## 0.1 Question 2: Objective Function

### 0.1.1 Question 2.a: Defining Objective Function

Suppose the hot sauces are sold at the same price: \$5 per liter.

What is the equation  $f(x)$  for the total revenue as a function of  $x_1$  and  $x_2$ ?

The function  $f(x)$  is called the objective function.

**SOLUTION**

$$f(x) = 5x_1 + 5x_2$$



### 0.1.2 Question 2.b: Direction of Steepest Increase

Since we want to maximize revenue, we want to increase our objective function as much as possible. Analogous to the minimization example given in a previous lecture, we can repeatedly move in the direction of function increase. In order to determine such direction, compute the gradient of  $f(x)$  at  $x = (0, 0)^T$ :

$$\nabla_x f(x) = \begin{pmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \end{pmatrix}$$

**SOLUTION**

$$\nabla_x f(x) = \begin{pmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \end{pmatrix} = \begin{pmatrix} \frac{\partial}{\partial x_1} 5x_1 + 5x_2 \\ \frac{\partial}{\partial x_2} 5x_1 + 5x_2 \end{pmatrix} = \begin{pmatrix} 5 \\ 5 \end{pmatrix}$$



## 0.2 Question 3: Putting Pieces Together

### 0.2.1 Question 3.a: Standard Form of a Linear Programming Problem

Write down the so-called the *standard form* of a linear programming problem:

$$\begin{aligned} \max_x & f(x) \\ \text{subject to} & Ax \leq b \end{aligned}$$

Specifically, write the objective as an inner product of two vectors:  $f(x) = c^T x$ , and write the constraint as a vector inequality involving a matrix-vector product:  $Ax \leq b$ , where  $A$  is a 4-by-2 matrix.

### SOLUTION

Write the objective as an inner product of two vectors:

$$f(x) = c^T x = \begin{bmatrix} 5 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

write the constraint as a vector inequality involving a matrix-vector product:  $Ax \leq b$ , where  $A$  is a 4-by-2 matrix.

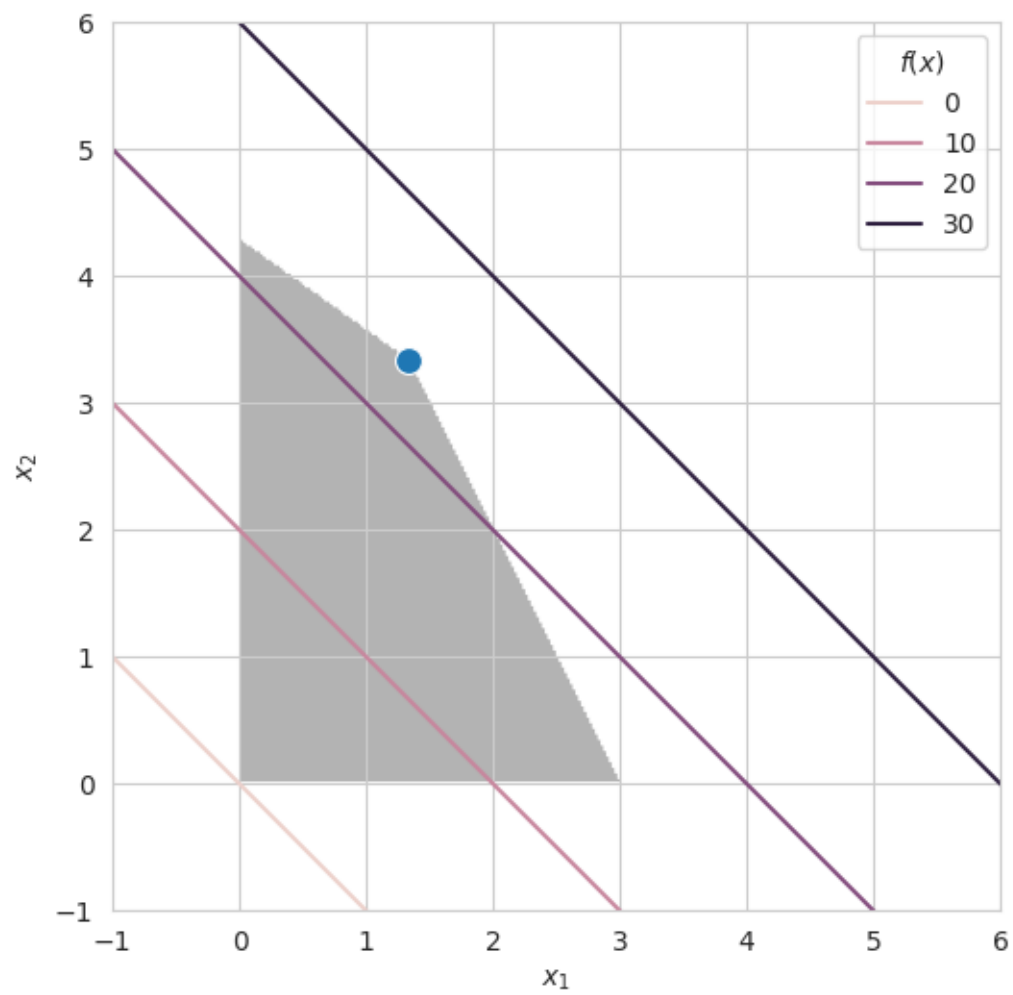
$$\begin{bmatrix} 5 & 7 \\ 4 & 2 \\ -1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \leq \begin{bmatrix} 30 \\ 12 \\ 0 \\ 0 \end{bmatrix}$$



## 0.2.2 Question 3.c: Plotting the optimal solution

```
In [16]: fig, ax = plt.subplots(figsize=(6, 6))
x1_grid, x2_grid = np.meshgrid(x1_line, x2_line)
ax.imshow(
    (
        (A1[0,0]*x1_grid + A1[0,1]*x2_grid <= b1[0]) & # Pepper constraints
        (A1[1,0]*x1_grid + A1[1,1]*x2_grid <= b1[1]) & # Vinegar constraints
        (A1[2,0]*x1_grid + A1[2,1]*x2_grid <= b1[2]) &
        (A1[3,0]*x1_grid + A1[3,1]*x2_grid <= b1[3]) # non-negativity constraints
    ),
    origin='lower',
    extent=(x1_grid.min(), x1_grid.max(), x2_grid.min(), x2_grid.max()),
    cmap="Greys", alpha = 0.3, aspect='equal'
)
sns.scatterplot(x='$x_1$', y='$x_2$', data=xstar1, ax=ax, s=100)
sns.lineplot(x='$x_1$', y='$x_2$', hue='$f(x)$', data=f_vals.reset_index(), ax=ax)
plt.xlim(-1, 6)
plt.ylim(-1, 6)
plt.show()
```

```
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead.
if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead.
if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead.
if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead.
if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead.
if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Use na_values=[np.inf, -np.inf] instead.
with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Use na_values=[np.inf, -np.inf] instead.
with pd.option_context('mode.use_inf_as_na', True):
```





### 0.2.3 Question 4.a: Define Constraints

To avoid eating the same foods, limit each food intake to be 2 or less. Also, one cannot consume less than zero servings. Furthermore, apply the nutritional constraints as specified in `nutritional_constraints.csv` (assume that the units are the same as food nutritional contents)

Note that a range constraints, e.g.,  $2000 \leq \text{total calories} \leq 2250$ , can be written as two constraints:  $\text{total calories} \leq 2250$  and  $-\text{total calories} \leq -2000$ . Hence, we can rewrite caloric intake constraints as

$$-(\text{calories in frozen broccoli})x_0 - (\text{calories in raw carrots})x_1 - \dots - (\text{calories in bean bacon soup, w/watr})x_{63} = -c^T x \leq -2000$$

$$\text{calories in frozen broccoli})x_0 + (\text{calories in raw carrots})x_1 + \dots + (\text{calories in bean bacon soup, w/watr})x_{63} = c^T x \leq 2250$$

where vector  $c$  contains calorie information for all 64 foods and  $x$  contains servings consumed of each food. Matrix  $U$  and vector  $w$  would be such that

$$U = \begin{pmatrix} -c^T \\ c^T \end{pmatrix} \text{ and } w = \begin{pmatrix} -2000 \\ 2250 \end{pmatrix},$$

and the matrix-vector inequality would be  $Ux \leq w$ . Range constraints of each food can be implemented similarly with identity matrices.

Denote nutritional content information from `foods` data frame as  $A$  and denote the `Min` and `Max` columns of `requirements` as vector  $b_L$  and  $b_U$ , respectively. Construct  $M$  and  $d$  in  $Mx \leq d$  using  $I$  (identity matrix),  $A$ ,  $b_L$ ,  $b_U$ , and other constants, so that all the range constraints are expressed in  $Mx \leq d$ . (This is a theory question. No coding is involved)

### SOLUTION

Given that limit each food intake to be 2 or less. Define the maximum servings constraint for each food item  $x_i (i = 1, 2, \dots, 64)$  such that  $x_i \leq 2$  i.e.

In matrix form:

$$[I_{64}] \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{64} \end{bmatrix} \leq \begin{bmatrix} 2 \\ 2 \\ \vdots \\ 2 \end{bmatrix}$$

Also we need to consider that the minimum servings constraint (Each food item  $x_i$  cannot be negative) s.t.  $x_i \geq 0$

In matrix form:

$$[-I_{64}] \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{64} \end{bmatrix} \leq \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Also data frame  $A$  have corresponding **Max** and **Min** columns of **requirements** as vector  $b_U$  and  $b_L$ , which is given by

$$Ax = b_U \quad -Ax = b_L$$

Then the aggregate range constraints can be expressed in  $Mx \leq d$

where

$$M = \begin{bmatrix} -I_{64} & 0 & \cdots & 0 \\ 0 & I_{64} & \ddots & \vdots \\ \vdots & \ddots & A & 0 \\ 0 & \cdots & 0 & -A \end{bmatrix}, x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{64} \end{bmatrix}, d = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 2 \\ \vdots \\ 2 \\ b_U \\ -b_L \end{bmatrix}$$

## 0.2.4 Question 4.d: Interpreting the Results

State the results in the context of the problem. How much of each food was consumed? List the foods and their calculated amounts. What is the total cost of feeding one soldier?

### SOLUTION

```
In [24]: names = []
         for i in range(foods.shape[0]):
             names.append(foods['Name'][i])

         servings = {name: servings for name, servings in zip(names, xstar2) if servings > 0.001}

         output = pd.DataFrame(list(servings.items()), columns=['Food', 'Servings'])
         output['Servings'] = output['Servings'].apply(lambda x: round(x, 2))
         total_cost = pd.DataFrame(['Food': 'Total Cost', 'Servings': f"${fstar2:.2f}"]) # total cost
         output = pd.concat([output, total_cost], ignore_index=True)

         print(output)
```

|    | Food                   | Servings |
|----|------------------------|----------|
| 0  | Frozen Broccoli        | 0.08     |
| 1  | Carrots, Raw           | 0.22     |
| 2  | Potatoes, Baked        | 2.0      |
| 3  | Wheat Bread            | 2.0      |
| 4  | White Bread            | 1.42     |
| 5  | Chocolate Chip Cookies | 2.0      |
| 6  | 3.3% Fat, Whole Milk   | 1.93     |
| 7  | Skim Milk              | 0.15     |
| 8  | Poached Eggs           | 0.25     |
| 9  | Malt-O-Meal, Choc      | 0.24     |
| 10 | White Rice             | 2.0      |
| 11 | Peanut Butter          | 2.0      |
| 12 | Popcorn, Air-Popped    | 2.0      |
| 13 | Total Cost             | \$1.25   |

There are total 13 different foods been consumed where the amounts are listed above. The total cost of feeding on soldier is \$1.25



### 0.2.5 (PSTAT 234) Question 5.a: Assignment Problem

Given a set  $\mathcal{S}$  of  $m$  people, a set  $\mathcal{D}$  of  $m$  tasks, and for each  $i \in \mathcal{S}, j \in \mathcal{D}$  a cost  $c_{ij}$  associated with assigning person  $i$  to task  $j$ , the assignment problem is to assign each person to one and only one task in such a manner that each task gets covered by someone and the total cost of the assignments is minimized. If we let

$$x_{ij} = \begin{cases} 1 & \text{if person } i \text{ is assigned task } j, \\ 0 & \text{otherwise,} \end{cases}$$

then the objective function can be written as

$$\text{minimize } \sum_{i \in \mathcal{S}} \sum_{j \in \mathcal{D}} c_{ij} x_{ij}.$$

Note that  $x_{ij}$  values are constrained to be 0 or 1. This is also a constraint. Suppose that, additionally, we would like to build in the constraints that 1. Each person is assigned exactly one task. 2. Every task gets covered by someone.

What equality constraints can you add?

#### SOLUTION

For each person, one is assigned exactly one task which is given by the following constraint:

$$\sum_{j \in \mathcal{D}} x_{ij} = 1, \quad \forall i \in \mathcal{S}$$

For each task, one is covered exactly by one person which is given by the following constraint:

$$\sum_{i \in \mathcal{S}} x_{ij} = 1, \quad \forall j \in \mathcal{D}$$

With the objective function:

$$\text{minimize } \sum_{i \in \mathcal{S}} \sum_{j \in \mathcal{D}} c_{ij} x_{ij}.$$



### 0.2.6 (PSTAT 234) Question 5.b: Transportation Problem

Transportation problem:  $i$  and  $j$  indicate pairs of locations, quantity of goods ( $x_{ij}$ ), supply ( $s_i$ ), demand ( $d_j$ ), transportation network ( $\mathcal{A}$ )

$$\begin{aligned} \min_{x=(x_{ij}) \geq 0} \quad & \sum_{(i,j) \in \mathcal{A}} c_{ij} x_{ij} \\ \text{subject to} \quad & \sum_{j:(i,j) \in \mathcal{A}} x_{ij} \leq s_i \quad \text{for all } i \\ & \sum_{i:(i,j) \in \mathcal{A}} x_{ij} \geq d_j \quad \text{for all } j \end{aligned}$$

Describe the assignment problem as a special case of the transportation problem. Draw analogy of supply network ( $\mathcal{A}$ ), supply ( $s_i$ ), demand ( $d_j$ ), and quantity of goods ( $x_{ij}$ ) in the assignment problem context.

#### SOLUTION

In the assignment problem, each person plays the role as a supplier with a supply of exactly 1. Also, each task plays the role as a demander with a demand of exactly 1.

Transportation Network  $\mathcal{A}$  represents the possible routes for transporting goods, here in the assignment problem a pair of  $(i, j) \in \mathcal{A}$  indicates that person  $i$  assigned to task  $j$ .

Quantity of goods  $x_{ij}$  represents the quantity of goods transported from location  $i$  to location  $j$ . As defined in the assignment problem,  $x_{ij}$  is a binary variable indicating whether person  $i$  is assigned to task  $j$  with

$$x_{ij} = \begin{cases} 1 & \text{if person } i \text{ is assigned task } j, \\ 0 & \text{otherwise,} \end{cases}$$

The cost  $c_{ij}$  in the assignment problem represents the cost of assigning person  $i$  to task  $j$ . The objective function is to minimize the total cost same as the assignment problem.

